IA aplicada a la detección de vulnerabilidades

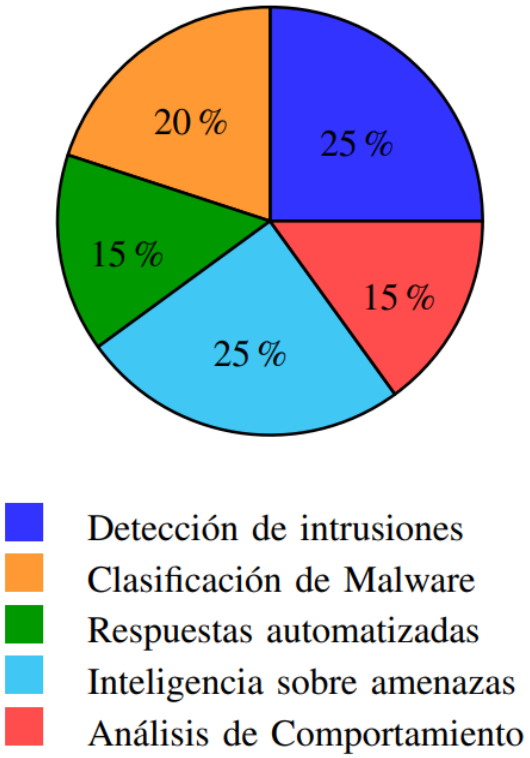
Metodología de la Investigación

Anthony Angel Briceño Quiroz

anthony.briceno@ucsp.edu.pe

La Crisis de las Vulnerabilidades

- Crecimiento: >25000 vulnerabilidades reportadas (2022)
- Problema: Herramientas tradicionales (SAST) = **Falsos Positivos**
- Necesidad: IA para la comprensión semántica, no solo reglas



Marco Teórico

Vulnerabilidad: Es un defecto o debilidad en el diseño, implementación o configuración de un sistema de software que puede ser explotado por un atacante para comprometer la seguridad del mismo.

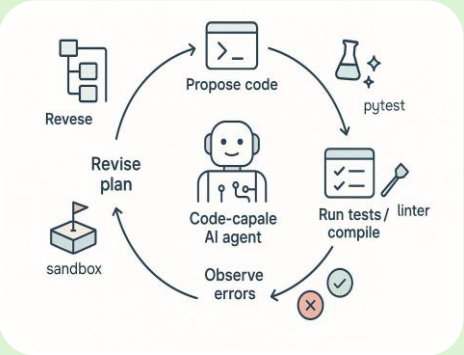
Aprendizaje Automático (ML): Es el punto de partida. Algoritmos que aprenden patrones estadísticos a partir de datos estructurados

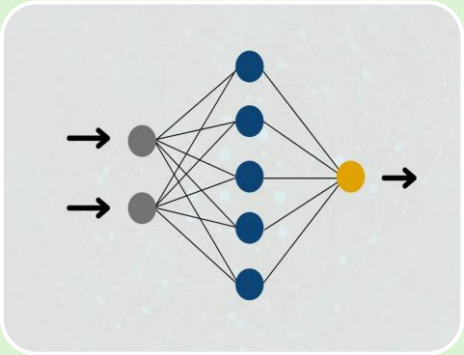
Graph Neural Network (GNN): El entendimiento de que el codigo no es lineal, si no una estructura (AST/CFG)

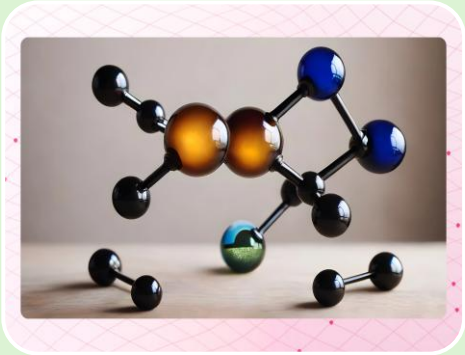
Transformers: El estado del arte. Atención masiva para entender el contexto global

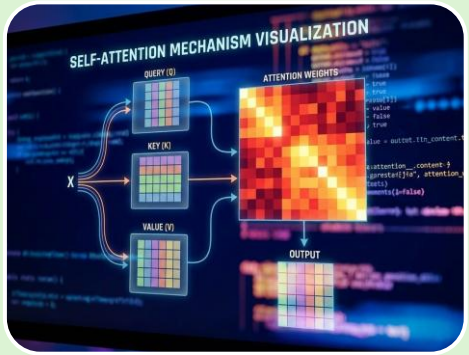
Aprendizaje Profundo (DL): El salto de a las Redes Neuronales Profundas, aquí entran las CNN y RNN/LSTM

Taxonomía Propuesta









ML y RL

- Pham
- Colin

DL Secuencial e Hibrido

- Wu
- Marchetto
- Han

GNN

- Liu
- Gear

Transformers

- Han
- Bahaa
- McCully

Dominio 1: ML Clásico y RL

- Pham et al (Fuzzing + RL)
  - Pro: Utiliza Deep-Q-Learning para priorizar semillas, logrando encontrar crashes 1.8x mas rapido que el fuzzing aleatorio.
  - CONTRA: La implementación en Python limita su velocidad de ejecución y solo se probó en binarios, faltando comparación con otros algoritmos de RL.
- Colin et al (Smart Contracts)
  - PRO: Su modelo MLP supera a herramientas estáticas (Mythril/Slither) con una tasa de falsos positivos muy baja (0.0125).
  - CONTRA: Depende totalmente de datos artificiales (inyección de fallos), lo que cuestiona su eficacia ante vulnerabilidades reales complejas no vistas en el entrenamiento.

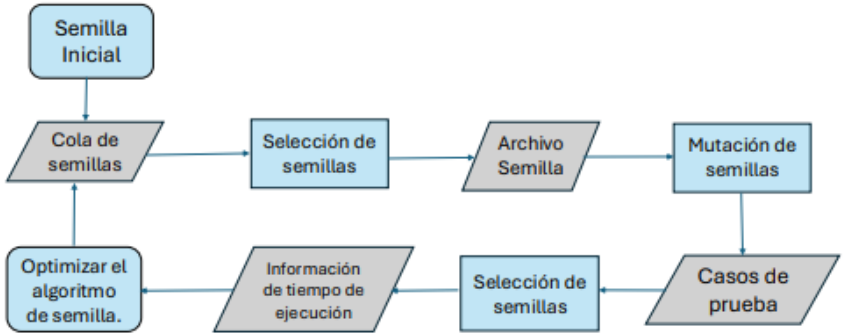


Figura 2: Proceso del *Fuzzing grey box. Figura adaptada y traducida de* [26].

Dominio 2: DL Secuencial e Hibrido

- Wu et al. (Binarios)
  - PRO: Modelo Bi-nivel (ELMo + BiGRU) que resuelve la polisemia (mismo código, distinto significado) y captura el contexto secuencial.
  - CONTRA: Pipeline complejo y sensible a la plataforma/compilador. Falta validación a gran escala en binarios heterogéneos reales.
- Marchetto et al. (Explicabilidad)
  - PRO: Introduce SHAP para no solo detectar, sino localizar la línea del fallo (Explicabilidad).
  - CONTRA: El modelo base (VulDeePecker) tiene un rendimiento bajo (F1 60%) y el conocimiento está limitado solo al código fuente sin contexto externo.
- Han et al. (CODE-SMASH)
  - PRO: La arquitectura siamesa jerárquica maneja mejor el código largo (+30% F1 vs previos).
  - CONTRAO: La arquitectura siamesa jerárquica maneja mejor el código largo (+30% F1 vs : Alto costo computacional y posible dependencia específica del dataset utilizado.

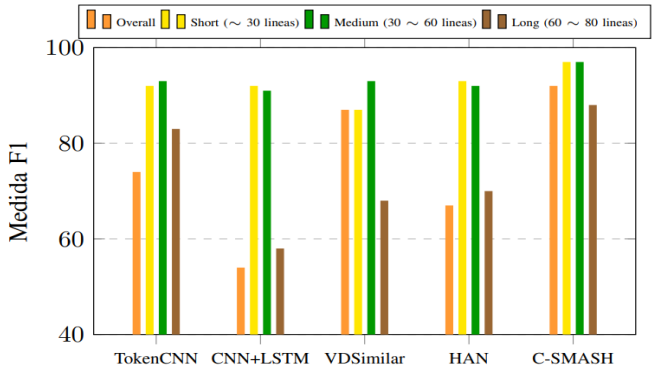


Figura 5: Comparativa de F1-Score según la longitud del código, en los diferentes grupos de el dataset Chromium. *Gráfico traducido de* [33].

Dominio 3: Modelos Basados en GNN

Concepto Clave: Convertir código en **AST** (Sintaxis) o **CFG** (Flujo) para encontrar fallos lógicos.

- Liu et al. (Blockchain):
  - PRO (El Híbrido Real): Combina GNN (generalización) con Reglas Expertas en la EVM, permitiendo bloquear ataques en tiempo real (88-90% de efectividad).
  - CONTRA: Las reglas expertas requieren ingeniería manual (no aprende solo) y añade un costo de ejecución (gas) en la Blockchain.
- Gear et al. (Poda de Grafos - SCEVD):
  - PRO: Usa Pruning (poda) semántica para eliminar el "ruido" del grafo, mejorando la precisión en datasets pequeños.
  - CONTRA: Probado solo en datos sintéticos (SARD Juliet), falta de realismo y requiere hardware potente para procesar los grafos.

Dominio 4: Modelos Basados en Transformers

Concepto Clave: Atención Global (entender todo el contexto a la vez).

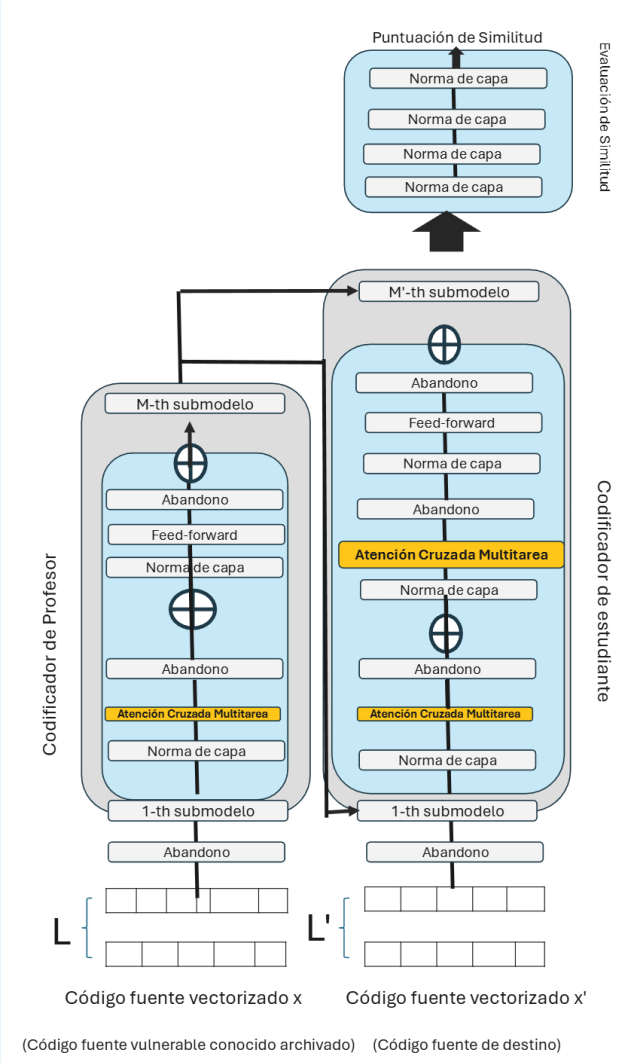
- Han et al. (X-Transformer)
  - **PRO:** Usa **Atención Cruzada** (Profesor-Alumno) y logra **95.45% F1 en Software Real** (Chromium/Debian). Además, es **4x-9x más rápido** en inferencia.
  - **CONTRA:** Dependencia de lenguajes específicos y falta análisis profundo de interpretabilidad.
- Bahaa et al. (DB-CBIL)
  - **PRO: Rendimiento Máximo:** Logra un F1 casi perfecto (**99.57%**) y Falsos Negativos de 0.0% gracias a mezclar DistilBERT con CNN+BiLSTM.
  - **CONTRA (La Trampa):** Evaluado puramente en **SARD Juliet (Sintético)**. Se limita a un solo tipo de vulnerabilidad (CWE-121) en C/C++.
- McCully et al. (TEDVIL)
  - **PRO:** Innova trabajando sobre **LLVM IR** (Código Intermedio), haciéndolo independiente del lenguaje fuente. GPT-2 superó a BERT aquí.
  - **CONTRA:** Requiere altos recursos computacionales y solo se probó en desbordamiento de búfer.

Rendimiento: Sintético vs Real

Cuadro II: Comparativa de Datasets utilizados y Rendimiento de Experimentos (Tabla II)

Dom.	Paper	Dataset / Entorno	Métricas y Resultados Clave
D1	Pham [26]	**Binarios Linux:** Binutils y Poppler 0.86.1 (Procesamiento PDF).	• **Cobertura:** +31 % vs Fuzzing aleatorio; +15 % vs Rifuzz. • **Velocidad:** Encuentra *crashes* 1.8x más rápido.
	Colin [31]	**Smart Contracts:** 100 inyectados + 100 reales (Solidity). Etiquetas SWC.	• **F1-Score:** 91.27 % • **Precisión:** 91.56 % • **FPR:** 0.0125 (Tasa de Falsos Positivos muy baja).
D2	Wu [32]	**Sintético:** Juliet v1.3 (CWE-121). **Real:** NDSS18 (Windows/Linux).	• **Juliet:** F1 $\approx$ 98.9 %. • **NDSS18-Whole:** F1 $\approx$ 87.7 %, AUC 0.92.
	Marchetto [16]	**Project KB (SAP):** 3,770 archivos Java (12 % vulnerables).	• **Detección:** JavaBERT F1 $\approx$ 88 %. • **Localización:** SHAP F1 $\approx$ 43 % (Error de 3–9 líneas).
	Han [33]	**Real (C/C++):** Chromium y Debian ($\sim$130k pares de funciones).	• **Chromium:** +30 % F1 vs métodos previos. • **Inferencia:** 2.43x más veloz que VDSimilar (<0.016s). • **Ventaja:** Manejo superior de código largo.
D3	Liu [34]	**ESC:** 40,932 contratos Ethereum (307k funciones).	• **Mejora:** +10 pts F1 sobre GCN estándar. • **Tiempo Real:** Bloquea 88–90 % de ataques (Reentrancy, Timestamp) en la EVM.
	Gear [35]	**SARD Juliet (C/C++):** 4 tipos CWE balanceados.	• **F1-Score:** 92–94 % (usando CPG + Chi-Squared). • **Impacto:** La poda (Pruning) mejoró +12 % el rendimiento.
D4	Han [36]	**Real:** Chromium, Debian. **Sintético:** Sun Dataset.	• **Chromium:** F1 $\approx$ 95.45 %, Acc $\approx$ 95.49 %. • **Debian:** F1 $\approx$ 95.15 %. • **Velocidad:** 4x–9x más rápido que TokenCNN.
	Bahaa [37]	**SARD Juliet:** 33k funciones C/C++ (Dataset Sintético).	• **F1-Score:** 99.57 % (El más alto reportado). • **Accuracy:** 99.81 %. • **FNR:** 0.0 % (Cero Falsos Negativos).
	McCully [38]	**Juliet v1.3:** Compilado a LLVM IR.	• **GPT-2 + LSTM:** Acc $\approx$ 92.5 %, F1 $\approx$ 89.7 %. • **Comparativa:** GPT-2 superó a BERT y RoBERTa en este contexto.

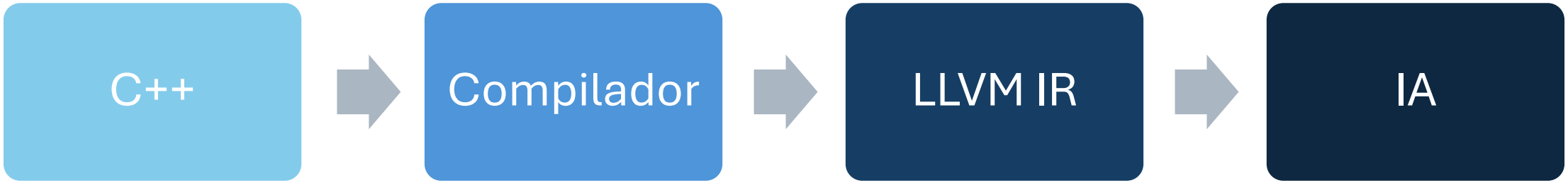
Leyenda: D1 ML/RL; **D2** DL Secuencial - Híbrido; **D3** GNN; **D4** Transformers.



Mayor generalización en software real

XTRANSFORMER

El Futuro -> TEDVIL y LLVM IR



Conclusiones

Ganador

El Transformer
X-Transformer
Por su robustez y velocidad

Hallazgo

- El rendimiento en datos sintéticos no garantiza éxito en la vida real

Reto

- Necesidad de datasets reales y modelos híbridos (Grafo + Transformer)

Referencias

- V.-H. Pham, D. Thi Thu Hien, N. Phuc Chuong, P. Thanh Thai, and P. The Duy, “A Coverage-Guided Fuzzing Method for Automatic Software Vulnerability Detection Using Reinforcement LearningEnabled Multi-Level Input Mutation,” IEEE Access, vol. 12, pp. 129 064–129 080, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10580893/
- L. S. H. Colin, P. M. Mohan, J. Pan, and P. L. K. Keong, “An Integrated Smart Contract Vulnerability Detection Tool Using Multi-Layer Perceptron on Real-Time Solidity Smart Contracts,” IEEE Access, vol. 12, pp. 23 549–23 567, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10430147/
- G. Wu and H. Tang, “Binary Code Vulnerability Detection Based on Multi-Level Feature Fusion,” IEEE Access, vol. 11, pp. 63 904– 63 915, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10160005/
- A. Marchetto, “Can explainability and deep-learning be used for localizing vulnerabilities in source code?” in Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024). Lisbon Portugal: ACM, Apr. 2024, pp. 110–119. [Online]. Available: https://dl.acm.org/doi/10.1145/3644032.3644448
- S. Han, H. Nam, J. Kang, K. Kim, S. Cho, and S. Lee, “CODE-SMASH: Source-Code Vulnerability Detection Using Siamese and Multi-Level Neural Architecture,” IEEE Access, vol. 12, pp. 102 492–102 504, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10606264/
- Z. Liu, M. Jiang, S. Zhang, J. Zhang, and Y. Liu, “A Smart Contract Vulnerability Detection Mechanism Based on Deep Learning and Expert Rules,” IEEE Access, vol. 11, pp. 77 990–77 999, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10190630/
- J. Gear, Y. Xu, E. Foo, P. Gauravaram, Z. Jadidi, and L. Simpson, “Software Vulnerability Detection Using Informed Code Graph Pruning,” IEEE Access, vol. 11, pp. 135 626–135 644, 2023. [Online]. Available: https://ieeexplore.ieee.org/document/10339268/
- S. Han, M. Kim, J. Kang, K. Kim, S. Lee, and S. Lee, “SimilarityBased Source Code Vulnerability Detection Leveraging Transformer Architecture: Harnessing Cross-Attention for Hierarchical Analysis,” IEEE Access, vol. 12, pp. 150 295–150 307, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10706239/
- A. Bahaa, A. E.-R. Kamal, H. Fahmy, and A. S. Ghoneim, “DB-CBIL: A DistilBert-Based Transformer Hybrid Model Using CNN and BiLSTM for Software Vulnerability Detection,” IEEE Access, vol. 12, pp. 64 446–64 460, 2024. [Online]. Available: https://ieeexplore.ieee.org/document/10517582/
- G. A. McCully, J. D. Hastings, and S. Xu, “TEDVIL: Leveraging Transformer-Based Embeddings for Vulnerability Detection in Lifted Code,” IEEE Access, vol. 13, pp. 76 894–76 913, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10980253/
- W. Stallings and L. Brown, Computer Security: Principles and Practice, 4th ed. Pearson, 2018.
- N. Mohamed, “Artificial intelligence and machine learning in cybersecurity: a deep dive into state-of-the-art techniques and future paradigms,” Knowledge and Information Systems, vol. 67, no. 8, pp. 6969–7055, Aug. 2025. [Online]. Available: https://link.springer.com/10.1007/s10115-025-02429-y